Supervyse OPF IPMI Developer's Guide

Revision 1.2

1/2025

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

1 IPMI OEM Command Implementation Guide 5
1.1 Introduction 5
1.2 How to add an IPMI OEM command 5
1.2.1 Source code of vendor-ipmi-oem 5
Define NetFn and Cmd 6
Implement command 7
Register command 8
Compile with Makefile 9
1.2.2 Recipe of vendor-ipmi-oem 10
Put the recipe into meta layer 10
Code Structure 10
Recipe: vendor-ipmi-oem.bb 11
1.2.3 Install package to Image 12
1.2.4 Add NetFn/Cmd into command whitelist (OPF G3 only) 13
1.2.5 NetFn Usage 15
1.3 Verification 15

2 OEM Module Development Guide 16
2.1 Introduction 16
2.2 How to add a OEM module 16
2.2.1 Source code of insyde-example-settings 17
Run as a D-Bus service 17
Expose a D-Bus property 17
Expose a D-Bus method 17
Listen to D-Bus signals 18
Compile with Makefile 19
2.2.2 Recipe of insyde-example-settings 20
Put the recipe into meta layer 20
Code Structure 20
Recipe: insyde-example-settings.bb 21
2.2.3 Install package to Image 22
2.3 Verification 23

Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2024-10-24	Roy Kuo	Brad Chang	Initial version
1.1	2024-12-19	Roy Kuo	Brad Chang	Forms adjustment
1.2	2025-01-22	Roy Kuo	Brad Chang	Release version

1 IPMI OEM Command Implementation Guide

1.1 Introduction

This page describes

- How to add an IPMI OEM Command
- Introduce the example code **vendor-ipmi-oem**

You need to have some background knowledge before studying this page:

- Yocto project
- IPMI Specification v2.0

1.2 How to add an IPMI OEM command

Steps to Implement an IPMI OEM Command:

1. Coding follows OpenBMC style command handler (C++ program)
2. Add a recipe for the IPMI OEM command in meta layer
3. Add the recipe to **IMAGE_INSTALL,** and the package will be installed into image.
4. Modify IPMI command whitelist

In the sections below, we use **vendor-ipmi-oem** as an example.

1.2.1 Source code of vendor-ipmi-oem

The source code file path depends on vendor and target platform:

- The source code file of CRBs in **OPF G3** SDK:
  - For Intel **ArcherCity** CRB

```
$(OPF_SDK)/meta-insyde/meta-intel/meta-archercity/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp
```

  - For AMD **Onyx** / **Quartz** CRBs

```
$(OPF_SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp
```

- For AMD **Turin** CRB

$(OPF_SDK)/meta-insyde/meta-amd/meta-turin/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

- For Nvidia **Falcon** / **Starship** / **MGX CG1** / **MGX C2** CRBs

$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

- For **ast2600evb**

$(OPF_SDK)/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

- The source code file of CRBs in **OPF G4** SDK:
  - For Intel **BeechnutCity** / **AvenueCity** CRBs

$(OPF_SDK)/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

  - For Nvidia **Oberon** CRB

$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

  - For **ast2600evb**

$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

  - For **ast2700evb**

$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

- The source code file of **new platform** created by customer:

$(OPF_SDK)/meta-base/meta-${new_platform}/recipes-platform/ipmi/vendor-ipmi-oem/src/vendor_ipmi_cmds.cpp

Define NetFn and Cmd

Add constant define for **NefFn** and **Cmd** for IPMI OEM command. Some NetFn and Cmd are already defined in phosphor-ipmi-host and intel-ipmi-oem.

More details are provided in the section below: NetFn Usage

Below is an example to define:

- NefFnVendorOem = 0x38
- CmdGetDummyResp = 0x01
- CmdGetUserPriv = 0x02

```
namespace ipmi
{
namespace vendor
{
constexpr NetFn netFnVendorOem = netFnOemFive; //0x38, define in phosphor-ipmi-host/include/ipmid/api-types.hpp

constexpr Cmd cmdGetDummyResp = 0x01;
constexpr Cmd cmdGetUserPriv = 0x02;
} // namespace vendor
```

Implement command

Below is an example of how to implement an IPMI OEM command:

```
// request data is user id, response user id and privilege
RspType<uint8_t,   // user id
        uint8_t    // privilege
        >
    ipmiGetUserPriv(Context::ptr ctx, uint8_t userId)
{
    static std::array<std::string, 6> ipmiPrivIndex = {
        "priv-reserved", // PRIVILEGE_RESERVED - 0
        "priv-callback", // PRIVILEGE_CALLBACK - 1
        "priv-user",     // PRIVILEGE_USER - 2
        "priv-operator", // PRIVILEGE_OPERATOR - 3
        "priv-admin",    // PRIVILEGE_ADMIN - 4
        "priv-custom"    // PRIVILEGE_OEM - 5
    };

    uint8_t privilege = 0;
    std::string userName;

    // log for check
    log<level::ERR>("IPMI OEM Command: ipmiGetUserPriv\n");

    // check user id is vaild
    if ((ipmiUserGetUserName(userId, userName) != ccSuccess) || userName.empty())
        return ipmi::responseParmOutOfRange();

    // get user privilege from phosphor-user-manager, or you can implement by the function ipmiUserGetPrivilegeAccess in phosphor-ipmi-host
    std::shared_ptr<sdbusplus::asio::connection> busp = getSdBus();
    std::string dbus_obj_path = USER_MANAGER_PATH;
    dbus_obj_path = dbus_obj_path + "/" + userName;
    try
    {
        Value vPrivString = getDbusProperty(*busp, USER_MANAGER_BUSNAME, dbus_obj_path, USER_MANAGER_USER_ATTRIBUTES_INTERFACE, "UserPrivilege");
        std::string privString = std::move(std::get<std::string>(vPrivString));

        auto iter = std::find(ipmiPrivIndex.begin(), ipmiPrivIndex.end(), privString);
        if (iter == ipmiPrivIndex.end())
        {
            if (privString != "")
            {
                log<level::ERR>("ipmiGetUserPriv: Error in converting to IPMI privilege\n",
                                entry("PRIV=%s", privString.c_str()));
            }
        }
        else
        {
            privilege = std::distance(ipmiPrivIndex.begin(), iter);
        }
    }
    catch (const sdbusplus::exception_t& e)
    {
        log<level::ERR>("ipmiGetUserPriv: unknown error\n");
        return responseUnspecifiedError();
    }

    return responseSuccess(userId, privilege);
}
```

Description of the number in the above picture:

1. Response data structure:
   - 1.1 Byte 1 represents user id and byte 2 represents privilege in the example
   - 1.2 It will be arranged into a series of bytes following the LSB format.
   - 1.3 If you want to design a command response with a variable length, please refer to phosphor-ipmi-host GetLanConfiguration command
2. IPMI command handler function name:
   - 2.1 It is **ipmiGetUserPriv** in the example
3. Context and request data:
   - 3.1 Byte 1 represents user id in the example
   - 3.2 **Context** is some information about this command request, e.g. channel, user, privilege...etc. Please see the structure in phosphor-ipmi-host message.hpp
   - 3.3 If you want to design a command request with a variable length, please refer to phosphor-ipmi-host SetLanConfiguration command
4. Complete code and response data:
   - 4.1 response data will be arranged into a series of bytes following the LSB format.
   - 4.2 return use API provided by phosphor-ipmi-host api-types.hpp

Register command

Programmers need to register the command handler with ipmid service

- Use **__attribute__((constructor))** for command register
- The **__attribute__((constructor))** makes the function execute when ipmid service use **dlopen** to the shared library which implement the command.

Below is an example: the ipmid service will invoke **resgisterVendorCmds** when opening the shared library.

```
static void registerVendorCmds() __attribute__((constructor));
```

registerHandler is the API to register an IPMI command handler.

See the details in the following path:
${OPF_SDK}/isc-src/ipmid/IPMI/Services/HostIpmid/host_ipmid_main.cpp

Below is an example of how to register the previous command:

```
static void registerVendorCmds(void)
{
    log<level::DEBUG>("register vendor ipmi oem commands");
    // Example: get user privilege
    registerHandler(prioOemBase, vendor::netFnVendorOem,
                    vendor::cmdGetUserPriv,
                    Privilege::User, ipmiGetUserPriv);

}
```

Description of the parameters for **registerHandler**:

- Priority:
  - It is prioOemBase in the example, see detail in phosphor-ipmi-host api-types.hpp

```
constexpr int prioOpenBmcBase = 10;
constexpr int prioOemBase = 20;
constexpr int prioOdmBase = 30;
constexpr int prioCustomBase = 40;
constexpr int prioMax = 50;
```

  - If more than one command register with the same NetFn/Cmd, the registration with lower priority will be ignored.
  - If the priorities are the same, the most recently registered command will be used.
- NetFn:
  - It is netFnVendorOem in the example and it is 0x38 in the previous implement.
- Cmd:
  - It is cmdGetUserPriv in the example and it is 0x02 in the previous implement.
- Privilege:
  - The user privilege limit for the command. It is privilege-user in the example. See details in the phosphor-ipmi-host api-types.hpp
- Command handler function name:
  - It is ipmiGetUserPriv in the example.

Compile with Makefile

Makefile is a description of how to compile source code. You may use other build systems (e.g. Cmake, Automake, Meson,etc.) if desired.

Below is an example:

- Compile all .cpp files in the folder
- Produce a shared library **liboemcmds**

```
#*****************************************************************************
#   Insyde CopyRight
#
#   Workfile:  make file for vendor-ipmi-oem
#
#   Abstract:  Makefile for building insyde ipmi oem command library
#              for user password expiry date
#
#*****************************************************************************/

CXX_SRCS := $(wildcard *.cpp)
CXX_OBJS := ${CXX_SRCS:cpp=o}

LIBS := liboemcmds.so

.PHONY: all
all: $(LIBS)

.PHONY: clean
clean:
    $(RM) $(CXX_OBJS) $(LIBS)

${LIBS} : ${CXX_OBJS}
    ${CXX} ${CXX_OBJS} -o ${LIBS} $(CFLAGS) $(LDFLAGS)
```

1.2.2 Recipe of vendor-ipmi-oem

A recipe is a collection of non-executable metadata used by BitBake to set variables or define additional build-time tasks. More details can be found in Yocto project

Put the recipe into meta layer

The recipe file path depends on vendor and target platform:

- The recipe file of CRBs in **OPF G3** SDK:
  - For Intel **ArcherCity** CRB

```
$(OPF_SDK)/meta-insyde/meta-intel/meta-archercity/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

  - For AMD **Onyx** / **Quartz** CRBs

```
$(OPF_SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

  - For AMD **Turin** CRB

```
$(OPF_SDK)/meta-insyde/meta-amd/meta-turin/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

  - For Nvidia **Falcon** / **Starship** / **MGX CG1** / **MGX C2** CRBs

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

  - For **ast2600evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

- The recipe file of CRBs in **OPF G4** SDK:
  - For Intel **BeechnutCity** / **AvenueCity** CRBs

```
$(OPF_SDK)/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

  - For Nvidia **Oberon** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

  - For **ast2600evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

  - For **ast2700evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2700evb/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

- The recipe file of **new platform** created by customer:

```
$(OPF_SDK)/meta-base/meta-${new_platform}/recipes-platform/ipmi/vendor-ipmi-oem.bb
```

Code Structure

Below is an example of the code structure from the previous implementation:

- **vendor-ipmi-oem.bb**: Recipe for vendor IPMI OEM command shared libary
- **vendor_ipmi_cmds.cpp**: Source code of IPMI OEM command
- **Makefile**: Makefile is used as the build system

```
vendor-ipmi-oem
└── src
    ├── Makefile
    └── vendor_ipmi_cmds.cpp
vendor-ipmi-oem.bb
```

Recipe: vendor-ipmi-oem.bb

Below is the recipe file vendor-ipmi-oem.bb:

```
SUMMARY = "Vendor IPMI OEM Command"
DESCRIPTION = "Vendor IPMI OEM Command"
LICENSE = "InsydeOpfRestrictedLicense"
LIC_FILES_CHKSUM = "file://${INSYDEBASE}/InsydeOpfRestrictedLicense;md5=da093b418598caaf318afbe1d3be63b1"
LICENSE_FLAGS = "InsydeOpfRestrictedLicense"
VERSION = "0.01"

DEPENDS += " systemd boost sdbusplus phosphor-logging phosphor-ipmi-host"

SRC_URI = "file://src/Makefile \
           file://src/vendor_ipmi_cmds.cpp \
          "

S = "${WORKDIR}"
TARGET_CXXFLAGS += " -std=c++20 -fPIC"
TARGET_LDFLAGS += " -lsystemd -lsdbusplus -lphosphor_logging -shared"
TARGET_LDFLAGS += " -L${STAGING_LIBDIR}/ipmid-providers -luserlayer"

do_compile() {
    make -C src
}

do_install () {
    install -d ${D}${libdir}/ipmid-providers/
    install -m 0755 ${S}/src/liboemcmds.so  ${D}${libdir}/ipmid-providers/liboemcmds.so
}

FILES:${PN}:append = " ${libdir}/ipmid-providers/"
```

Description of the number in the above picture:

1 Description of recipe and license
   1.1 Description is "Vendor IPMI OEM Command"
   1.2 Use Insyde license in the example
2 Recipe dependency
   2.1 Example recipe is depnends-on "systemd boost ... etc"
   2.2 **phosphor-ipmi-host** is required
3 Source code files
   3.1 Makefile and vendor_ipmi_cmds.cpp are source code files in the example
4 Pass the compile flags to Makefile
   4.1 pass S=${WORKDIR} and compile/link flag in the example
   4.2 **S** and **${WORKDIR}** are variable use in Yocto. More details can be found in Yocto project
5 How to compile source code
   5.1 Directly start by running make in the example
6 How to install library
   6.1 Create directory and install shared library liboemcmds.so in the example
   6.2 ${libdir}/ipmid-providers is the path where command handler shared library located.

1.2.3 Install package to Image

Programmers need to add the recipe to the **IMAGE_INSTALL** variable, and then the recipe will be executed during the firmware build process.

The recipe file path depends on vendor and target platform:

- The recipe file of CRBs in **OPF G3** SDK:
  - For Intel **ArcherCity** CRB

```
$(OPF_SDK)/meta-insyde/meta-intel/meta-archercity/recipes-platform/images/intel-platforms.bbappend
```

  - For AMD **Onyx** / **Quartz** CRBs

```
$(OPF_SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/images/obmc-phosphor-image.bbappend
```

  - For AMD **Turin** CRB

```
$(OPF_SDK)/meta-insyde/meta-amd/meta-turin/recipes-platform/images/obmc-phosphor-image.bbappend
```

  - For Nvidia **Falcon** / **Starship** / **MGX CG1** / **MGX C2** CRBs

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/images/obmc-phosphor-image.bbappend
```

  - For **ast2600evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/images/obmc-phosphor-image.bbappend
```

- The recipe file of CRBs in **OPF G4** SDK:
  - For Intel **BeechnutCity** / **AvenueCity** CRBs

```
$(OPF_SDK)/meta-insyde/meta-intel/meta-bhs/recipes-platform/images/intel-platforms.bbappend
```

  - For Nvidia **Oberon** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/images/obmc-phosphor-image.bbappend
```

  - For **ast2600evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/images/obmc-phosphor-image.bbappend
```

  - For **ast2700evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/images/obmc-phosphor-image.bbappend
```

- The recipe file of **new platform** created by customer:

```
$(OPF_SDK)/meta-base/meta-${new_platform}/recipes-platform/images/obmc-phosphor-image.bbappend
```

Below is an example:

- ```
  IMAGE_INSTALL:append = " vendor-ipmi-oem"
  ```

1.2.4 Add NetFn/Cmd into command whitelist (OPF G3 only)

In **OPF G3**:

The whitelist determines which commands should use the OpenBMC-style command handler.

In **OPF G4**:

The whitelist is not necessary; you can skip this section.

The source code file path:

```
${OPF_SDK}/isc-src/ipmid/IPMI/Services/CmdHandler/CmdTableObmc.c
```

Below is an example add previous command to whitelist:

```
    {NETFN_CMD_TBL(NETFN_INTEL_OEM_PLATFORM, CMD_SET_PFR_PROVISIONING_STATUS), 0}, // 0xF1
    {NETFN_CMD_TBL(NETFN_INTEL_OEM_PLATFORM, CMD_GET_PFR_PROVISIONING_LOCK), 0}, // 0xF2
    {NETFN_CMD_TBL(NETFN_INTEL_OEM_PLATFORM, CMD_SET_PFR_PROVISIONING_LOCK), 0}, // 0xF3
    // NETFN_INSYDE_OEM
    {NETFN_CMD_TBL(NETFN_INSYDE_OEM, CMD_SET_USER_PASSWD_EXPIRY_DATE), 0}, //0x01
    {NETFN_CMD_TBL(NETFN_INSYDE_OEM, CMD_GET_USER_PASSWD_EXPIRY_DATE), 0}, //0x02
    {NETFN_CMD_TBL(NETFN_INSYDE_OEM, CMD_RM_VNC_SESSION), 0}, //0x03
    // NETFN_VENDOR_OEM
    {NETFN_CMD_TBL(NETFN_VENDOR_OEM, CMD_GET_DUMMY_RESP), 0}, //0x01
    {NETFN_CMD_TBL(NETFN_VENDOR_OEM, CMD_GET_USER_PRIV), 0}, //0x02
    //end of list
    {0xffff, 0}
};
```

Description:

- Use the marco **NETFN_CMD_TBL** with format NETFN_CMD_TBL(NetFm, Cmd) to add the OEM command to the whitelist.
- The second variable of the table should be reserved as **0**.

See the constant define for NefFn and Cmd in file

```
${OPF_SDK}/isc-src/ipmid/IPMI/Include/IpmiCmdDefs.h
```
```

```
enum IPMI_NETFN {
    NETFN_CHASSIS = 0x00,
    NETFN_BRIDGE = 0x02,
    NETFN_SENSOR = 0x04,
    NETFN_APP = 0x06,
    NETFN_FIRMWARE = 0x08,
    NETFN_STORAGE = 0x0A,
    NETFN_TRANSPORT = 0x0C,
    NETFN_GROUP_EXTENSION = 0x2C,
    NETFN_NODE_MANAGER = 0x2E,

    /* OEM-specific Netfunction */
    NETFN_INTEL_OEM_GENERAL = 0x30,
    NETFN_INTEL_OEM_PLATFORM = 0x32,
    NETFN_INTEL_OEM_APP = 0x3E,

#if (defined(OPF_NVIDIA) && (OPF_NVIDIA != 0))
    NETFN_NVIDIA_OEM_POST = 0x34, // openbmc nvidia use 0x30
    NETFN_NVIDIA_OEM_GLOBAL = 0x36, // openbmc nvidia use 0x32
    NETFN_NVIDIA_OEM_NV = 0x3C,
#endif
    NETFN_INSYDE_OEM = 0x3A,
    NETFN_VENDOR_OEM = 0x38,
};
```

```
enum IPMI_NETFN_VENDOR_OEM_CMD {
    CMD_GET_DUMMY_RESP = 0x01,
    CMD_GET_USER_PRIV = 0x02,
};
```

1.2.5 NetFn Usage

Following the **IPMI Specification v2.0, Table 5 (Network Function Codes)**, NetFn codes from 30h to 3Fh are vendor-specific.

The following NetFn are already used:

- Intel NetFn
  - NetFn general = 30h
  - NetFn platform = 32h
  - NetFn app = 3Eh
- Nvidia NetFn (Nvidia platform only)
  - OPF uses Intel NetFn defined commands in Nvidia, and changes the default Nvidia NetFn to avoid command conflicts.
  - NetFn OEM post = 34h (The original value is 30h)
  - NetFn OEM global = 36h (The original value is 32h)
  - NetFn OEM NV = 3Ch
  - NetFn OEM Eight = 3Eh
- Insyde NetFn OEM = 3Ah

1.3 Verification

To verify the IPMI OEM command using ipmitool in the BMC console:

```
root@nvidia-obmc:~# ipmitool raw 0x38 0x02 0x01
 01 04
```

2 OEM Module Development Guide

2.1 Introduction

This page describes

- How to add an OEM module
- Introduce the example code **insyde-example-settings**

You need to have some background knowledge before studying this page:

- Yocto project
- D-Bus

2.2 How to add a OEM module

Steps to add an OEM module:

1. Implement an OEM module
2. Add a recipe for the OEM module in meta layer
3. Add the recipe to **IMAGE_INSTALL,** and the package will be installed into image

In the sections below, we use **insyde-example-settings** as an example. Which is a **D-Bus** service.

Note:

- **D-Bus** is a message-oriented middleware mechanism that allows communication between multiple processes running concurrently on the same machine.
- Simply, D-Bus is a IPC just like **pipe**, **shared memory**, **message queue** ...etc.

2.2.1 Source code of insyde-example-settings

The source code file path:

- In **OPF G3** SDK:

${OPF_SDK}/meta-insyde/meta-common/meta-spv/recipes-phosphor/settings/insyde-example-settings/src/insyde_example_settings_main.cpp

- In **OPF G4** SDK:

${OPF_SDK}/meta-insyde/meta-common/meta-core/recipes-phosphor/settings/insyde-example-settings/src/insyde_example_settings_main.cpp

Run as a D-Bus service

A D-Bus service needs **D-Bus service name**, **D-Bus object path** and **D-Bus interface** for initialization.

Below is an example:

- D-Bus service name is **xyz.openbmc_project.Insyde.Example.Settings**
- D-Bus object path is **/xyz/openbmc_project/insyde/example/settings**

```
// initial boost asio and sdbus
boost::asio::io_context io;
auto bus = std::make_shared<sdbusplus::asio::connection>(io);

// example about dbus service
// dbus service name: xyz.openbmc_project.Insyde.Example.Settings
// dbus object path:  /xyz/openbmc_project/insyde/example/settings
// dbus interface:    xyz.openbmc_project.insyde.example.settings
bus->request_name("xyz.openbmc_project.Insyde.Example.Settings");
sdbusplus::asio::object_server objServer(bus);
auto example_iface = objServer.add_interface("/xyz/openbmc_project/insyde/example/settings", "xyz.openbmc_project.insyde.example.settings");
```

- D-Bus interface is **xyz.openbmc_project.insyde.example.settings**

Expose a D-Bus property

A **D-Bus property** is the service expose data on D-Bus. Other processes can retrieve the data via D-Bus or listen to signals when property changes.

Below is an example that implements the D-Bus property **Version** with read/write permissions.

```
// example dbus property
// property name: Version
// functional: show insyde-example-settings version (read/write)
example_iface->register_property("Version", INSYDE_EXAMPLE_SETTINGS_VERSION, sdbusplus::asio::PropertyPermission::readWrite);
```

Expose a D-Bus method

A **D-Bus method** is a service that expose a function on D-Bus. Other processes can call the function via D-Bus.

Below is an example that implements the D-Bus method **GetIdentifyLedState** to retrieve the current identify LED state from the phosphor-led-sysfs service.

```
// example dbus method
// method name: GetIdentifyLedState
// functional: get current identify LED state from phosphor-led-sysfs
example_iface->register_method("GetIdentifyLedState",
    [bus](){
        auto method =
            bus->new_method_call("xyz.openbmc_project.LED.Controller.identify", "/xyz/openbmc_project/led/physical/identify", "org.freedesktop.DBus.Properties", "Get");
        method.append("xyz.openbmc_project.Led.Physical", "State");

        try {
            auto reply = bus->call(method);
            std::variant<std::string> vsLedState;
            reply.read(vsLedState);
            std::string sLedState = std::get<std::string>(vsLedState);
            return sLedState;
        } catch (...) {
            log<level::INFO>("Fail to get identify led status");
            return LED_STATE_OFF;
        }
    });
```

Listen to D-Bus signals

A D-Bus match is a service that listens to certain signals on D-Bus. Usually, it is a signal indicating a property change or an interface addition/removal.

Below is an example with:

- Register a D-Bus property **IdentifyLedState** with read only permission.
- Use the match **identityLedPropChangeMatch** to synchronize the identify LED state when the LED state changes in phosphor-led-sysfs
- Retrieve the current LED state after adding the match to avoid an incorrect initial state

```
// example dbus match
// register a dbus propertty IdentifyLedState and sync the identify LED state with phosphor-led-sysfs
// property name: IdentifyLedState
// functional: sync the identify LED state with phosphor-led-sysfs (read only)
example_iface->register_property("IdentifyLedState", LED_STATE_OFF);

constexpr const char* IDENTIFY_LED_STATE_FILTER = "type='signal',interface='org.freedesktop.DBus.Properties'"
                                                  ",path='/xyz/openbmc_project/led/physical/identify',member='PropertiesChanged'"
                                                  ",arg0='xyz.openbmc_project.Led.Physical'";
sdbusplus::bus::match_t identityLedPropChangeMatch(*bus, IDENTIFY_LED_STATE_FILTER,
    [example_iface](sdbusplus::message::message& message) {
        using DbusMtmProperties =
            std::vector<std::pair<std::string, std::variant<std::string>>>;
        try {
            std::string intfName;
            DbusMtmProperties chProperties;
            message.read(intfName, chProperties);

            for (const auto& prop : chProperties)
            {
                if ("State" == prop.first) {
                    std::string sLedState = std::get<std::string>(prop.second);
                    example_iface->set_property("IdentifyLedState", sLedState);
                }
            }
        } catch (...) {
            log<level::INFO>("Fail to sync identify led status");
        }
    });
// Note: get current state after add match to avoid initial state is incorrect
auto method =
    bus->new_method_call("xyz.openbmc_project.LED.Controller.identify", "/xyz/openbmc_project/led/physical/identify", "org.freedesktop.DBus.Properties", "Get");
method.append("xyz.openbmc_project.Led.Physical", "State");

try {
    auto reply = bus->call(method);
    std::variant<std::string> vsLedState;
    reply.read(vsLedState);
    std::string sLedState = std::get<std::string>(vsLedState);
    example_iface->set_property("IdentifyLedState", sLedState);
} catch (...) {
    log<level::INFO>("Fail to get identify led status");
}
```

Compile with Makefile

Makefile is a description of how to compile source code. You may use other build systems (e.g. Cmake, Automake, Meson,etc.) if desired.

Below is an example:

- Compile all .cpp files in folder
- Produce a service **insyde-example-settings**

```
#*****************************************************************************
#   Insyde CopyRight
#
#   Workfile:  make file for insyde exmaple settings service
#
#   Abstract:  Makefile for building insyde example settings service
#
#*****************************************************************************/

CXX_SRCS := $(wildcard *.cpp)
CXX_OBJS := ${CXX_SRCS:cpp=o}

PROGS := insyde-example-settings

.PHONY: all
all: $(PROGS)

.PHONY: clean
clean:
    $(RM) $(CXX_OBJS) $(PROGS)

${PROGS} : ${CXX_OBJS}
    ${CXX} ${CXX_OBJS} -o ${PROGS} $(CFLAGS) $(LDFLAGS)
```

2.2.2 Recipe of insyde-example-settings

Put the recipe into meta layer

The recipe file path:

- In **OPF G3** SDK:

```
${OPF_SDK}/meta-insyde/meta-common/meta-spv/recipes-phosphor/settings/insyde-example-settings.bb
```

- In **OPF G4** SDK:

```
${OPF_SDK}/meta-insyde/meta-common/meta-core/recipes-phosphor/settings/insyde-example-settings.bb
```

Code Structure

Below is an example of the code structure for insyde-example-settings:

- **insyde-example-settings.bb**: Recipe for the service
- **xyz.openbmc_project.Insyde.Example.Settings.service**: It is systemd service unit
- **insyde_example_settings_main.cpp**: Source code of the service
- **Makefile:** Makefile is used as the build system

```
settings
├── insyde-example-settings
│   ├── src
│   │   ├── Makefile
│   │   └── insyde_example_settings_main.cpp
│   └── xyz.openbmc_project.Insyde.Example.Settings.service
└── insyde-example-settings.bb
```

Recipe: insyde-example-settings.bb

Below is the recipe file insyde-example-settings.bb:

```
SUMMARY = "Insyde Example Settings Service"
DESCRIPTION = "Service design to modify run-time settings for Insyde"
LICENSE = "CLOSED"
LIC_FILES_CHKSUM = ""

DEPENDS += " systemd boost sdbusplus phosphor-logging"

inherit obmc-phosphor-systemd

SYSTEMD_SERVICE:${PN} = "xyz.openbmc_project.Insyde.Example.Settings.service"

SRC_URI = "file://src/Makefile \
           file://src/insyde_example_settings_main.cpp \
          "

S = "${WORKDIR}"
TARGET_CXXFLAGS += " -std=c++20"
TARGET_LDFLAGS += " -lsystemd -lsdbusplus -lphosphor_logging"

do_compile() {
    make -C src
}

do_install () {
    install -d ${D}${bindir}
    install -m 0755 ${S}/src/insyde-example-settings ${D}${bindir}/insyde-example-settings
}
```

Description of the number in the above picture:

1 Description of recipe and license
  1.1 Description is "Service design to modify run-time settings for Insyde"
  1.2 No license in the example
2 Recipe dependency
  2.1 Example recipe is depnends-on "systemd boost ... etc"
3 Source code files
  3.1 Makefile and insyde_example_settings_main.cpp are source code files in the example
4 Pass the compile flags to Makefile
  4.1 pass S=${WORKDIR} and compile/link flag in example
  4.2 **S** and **${WORKDIR}** are variable use in Yocto. More details can be found in Yocto project
5 How to compile source code
  5.1 Directly start by running make in the example
6 How to install service
  6.1 Create directory and install binary file insyde-example-settings in example

2.2.3 Install package to Image

Programmers need to add the recipe to the IMAGE_INSTALL variable, and then the recipe will be executed during the firmware build process.

The recipe file path:

- In **OPF G3** SDK:

  ${OPF_SDK}/meta-insyde/meta-common/meta-share/classes/insyde-common-image.bbclass

- In **OPF G4** SDK:

  ${OPF_SDK}/meta-insyde/meta-common/meta-core/classes/insyde-common-image.bbclass

Below is an example that adds insyde-example-settings to IMAGE_INSTALL variable:

```
IMAGE_INSTALL:remove = " phosphor-hwmon"
IMAGE_INSTALL:append = " dbus-sensors"
IMAGE_INSTALL:append = " tzdata"
IMAGE_INSTALL:append = " tzcode"
IMAGE_INSTALL:append = " krb5"
IMAGE_INSTALL:append = " krb5validate"
IMAGE_INSTALL:append = " cyrus-sasl"
IMAGE_INSTALL:append = " openpam"
IMAGE_INSTALL:append = " openldapclient"
IMAGE_INSTALL:append = " insyde-example-settings"
```

2.3 Verification

- Display the D-Bus interface.

Test command and result:

```
busctl introspect xyz.openbmc_project.Insyde.Example.Settings
/xyz/openbmc_project/insyde/example/settings xyz.openbmc_project.insyde.example.settings
```

```
root@nvidia-obmc:~# busctl introspect xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings
NAME                                TYPE      SIGNATURE RESULT/VALUE                            FLAGS
.GetIdentifyLedState                method    -         s                                       -
.IdentifyLedState                   property  s         "xyz.openbmc_project.Led.Physical.Act... emits-change
.Version                            property  s         "1.0"                                   emits-change writable
```

- Read and Write the D-Bus property **Version**.

Test command and result:

```
busctl introspect xyz.openbmc_project.Insyde.Example.Settings
/xyz/openbmc_project/insyde/example/settings xyz.openbmc_project.insyde.example.settings

busctl set-property xyz.openbmc_project.Insyde.Example.Settings
/xyz/openbmc_project/insyde/example/settings xyz.openbmc_project.insyde.example.settings Version
s "2.0"
```

```
root@nvidia-obmc:~# busctl get-property xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings Version
s "1.0"
root@nvidia-obmc:~# busctl set-property xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings Version s "2.0"
root@nvidia-obmc:~# busctl get-property xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings Version
s "2.0"
```

- Call the method **GetIdentifyLedState** and change identify LED state to verify.

Test command and result:

```
busctl call xyz.openbmc_project.Insyde.Example.Settings
/xyz/openbmc_project/insyde/example/settings xyz.openbmc_project.insyde.example.settings
GetIdentifyLedState

busctl set-property xyz.openbmc_project.LED.Controller.identify
/xyz/openbmc_project/led/physical/identify xyz.openbmc_project.Led.Physical State s
xyz.openbmc_project.Led.Physical.Action.On
```

```
root@bmc-maca4bf018d03a8:~# busctl call xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings GetIdentifyLedState
s "xyz.openbmc_project.Led.Physical.Action.Off"
root@bmc-maca4bf018d03a8:~# busctl set-property xyz.openbmc_project.LED.Controller.identify /xyz/openbmc_project/led/physical/identify \
>  xyz.openbmc_project.Led.Physical State s xyz.openbmc_project.Led.Physical.Action.On
root@bmc-maca4bf018d03a8:~# busctl call xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings GetIdentifyLedState
s "xyz.openbmc_project.Led.Physical.Action.On"
```

- Check if the D-Bus match works by getting the property **IdentifyLedState** and then changing the identify LED state to verify.

Test command and result:

```
busctl get-property xyz.openbmc_project.Insyde.Example.Settings
/xyz/openbmc_project/insyde/example/settings xyz.openbmc_project.insyde.example.settings
IdentifyLedState

busctl set-property xyz.openbmc_project.LED.Controller.identify
/xyz/openbmc_project/led/physical/identify xyz.openbmc_project.Led.Physical State s
xyz.openbmc_project.Led.Physical.Action.On
```

```
root@bmc-maca4bf018d03a8:~# busctl get-property xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings IdentifyLedState
s "xyz.openbmc_project.Led.Physical.Action.Off"
root@bmc-maca4bf018d03a8:~# busctl set-property xyz.openbmc_project.LED.Controller.identify /xyz/openbmc_project/led/physical/identify \
> xyz.openbmc_project.Led.Physical State s xyz.openbmc_project.Led.Physical.Action.On
root@bmc-maca4bf018d03a8:~# busctl get-property xyz.openbmc_project.Insyde.Example.Settings /xyz/openbmc_project/insyde/example/settings \
> xyz.openbmc_project.insyde.example.settings IdentifyLedState
s "xyz.openbmc_project.Led.Physical.Action.On"
```